

20 Java Coding Practice Problems

HighRadius / Placement Preparation — Simple Interview-Friendly Solutions

The code is intentionally kept simple so you can understand, write, and reproduce it during a coding round.

1. Move Zeroes to End

```
public class Main {
    static void moveZeroes(int[] arr) {
        int index = 0;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] != 0) {
                arr[index] = arr[i];
                index++;
            }
        }

        while (index < arr.length) {
            arr[index] = 0;
            index++;
        }
    }

    public static void main(String[] args) {
        int[] arr = {0, 1, 0, 3, 12};

        moveZeroes(arr);

        for (int x : arr) {
            System.out.print(x + " ");
        }
    }
}
```

2. Leader in an Array

```
public class Main {
    static void leaders(int[] arr) {
        int maxRight = arr[arr.length - 1];

        System.out.print(maxRight + " ");

        for (int i = arr.length - 2; i >= 0; i--) {
            if (arr[i] > maxRight) {
                maxRight = arr[i];
                System.out.print(maxRight + " ");
            }
        }
    }

    public static void main(String[] args) {
        int[] arr = {16, 17, 4, 3, 5, 2};

        leaders(arr);
    }
}
```

3. Container With Most Water

```
public class Main {
    static int maxWater(int[] height) {
        int left = 0;
        int right = height.length - 1;
        int max = 0;

        while (left < right) {
            int width = right - left;
            int water = Math.min(height[left], height[right]) * width;

            max = Math.max(max, water);

            if (height[left] < height[right]) {
                left++;
            } else {
                right--;
            }
        }

        return max;
    }

    public static void main(String[] args) {
        int[] height = {1, 8, 6, 2, 5, 4, 8, 3, 7};

        System.out.println(maxWater(height));
    }
}
```

4. Segregate 0s, 1s, and 2s

```
public class Main {
    static void sort012(int[] arr) {
        int low = 0;
        int mid = 0;
        int high = arr.length - 1;

        while (mid <= high) {
            if (arr[mid] == 0) {
                int temp = arr[low];
                arr[low] = arr[mid];
                arr[mid] = temp;

                low++;
                mid++;
            } else if (arr[mid] == 1) {
                mid++;
            } else {
                int temp = arr[mid];
                arr[mid] = arr[high];
                arr[high] = temp;

                high--;
            }
        }
    }

    public static void main(String[] args) {
        int[] arr = {2, 0, 2, 1, 1, 0};

        sort012(arr);

        for (int x : arr) {
            System.out.print(x + " ");
        }
    }
}
```

5. Equilibrium Index

```
public class Main {
    static int equilibriumIndex(int[] arr) {
        int total = 0;

        for (int x : arr) {
            total += x;
        }

        int leftSum = 0;

        for (int i = 0; i < arr.length; i++) {
            int rightSum = total - leftSum - arr[i];

            if (leftSum == rightSum) {
                return i;
            }

            leftSum += arr[i];
        }

        return -1;
    }

    public static void main(String[] args) {
        int[] arr = {1, 3, 5, 2, 2};

        System.out.println(equilibriumIndex(arr));
    }
}
```

6. Product of Array Except Self

```
public class Main {
    static int[] productExceptSelf(int[] arr) {
        int n = arr.length;
        int[] result = new int[n];

        int product = 1;

        // Product of elements to the left
        for (int i = 0; i < n; i++) {
            result[i] = product;
            product *= arr[i];
        }

        product = 1;

        // Product of elements to the right
        for (int i = n - 1; i >= 0; i--) {
            result[i] *= product;
            product *= arr[i];
        }

        return result;
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4};

        int[] result = productExceptSelf(arr);

        for (int x : result) {
            System.out.print(x + " ");
        }
    }
}
```

7. Majority Element — Moore's Voting Algorithm

```
public class Main {
    static int majorityElement(int[] arr) {
        int candidate = 0;
        int count = 0;

        for (int x : arr) {
            if (count == 0) {
                candidate = x;
            }

            if (x == candidate) {
                count++;
            } else {
                count--;
            }
        }

        return candidate;
    }

    public static void main(String[] args) {
        int[] arr = {2, 2, 1, 1, 1, 2, 2};

        System.out.println(majorityElement(arr));
    }
}
```

8. Minimum Platforms Required

```
import java.util.Arrays;

public class Main {
    static int minPlatforms(int[] arrival, int[] departure) {
        Arrays.sort(arrival);
        Arrays.sort(departure);

        int i = 0;
        int j = 0;
        int platforms = 0;
        int maxPlatforms = 0;

        while (i < arrival.length && j < departure.length) {
            if (arrival[i] <= departure[j]) {
                platforms++;
                maxPlatforms = Math.max(maxPlatforms, platforms);
                i++;
            } else {
                platforms--;
                j++;
            }
        }

        return maxPlatforms;
    }

    public static void main(String[] args) {
        int[] arrival = {900, 940, 950, 1100, 1500, 1800};
        int[] departure = {910, 1200, 1120, 1130, 1900, 2000};

        System.out.println(minPlatforms(arrival, departure));
    }
}
```

9. Wave Array

```
public class Main {
    static void wave(int[] arr) {
        for (int i = 0; i < arr.length - 1; i += 2) {
            int temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5, 6};

        wave(arr);

        for (int x : arr) {
            System.out.print(x + " ");
        }
    }
}
```

10. Subarray With Given Sum

```
public class Main {
    static void findSubarray(int[] arr, int target) {
        int left = 0;
        int sum = 0;

        for (int right = 0; right < arr.length; right++) {
            sum += arr[right];

            while (sum > target && left <= right) {
                sum -= arr[left];
                left++;
            }

            if (sum == target) {
                System.out.println("Start index: " + left);
                System.out.println("End index: " + right);
                return;
            }
        }

        System.out.println("No subarray found");
    }

    public static void main(String[] args) {
        int[] arr = {1, 4, 20, 3, 10, 5};

        findSubarray(arr, 33);
    }
}
```

11. First Non-Repeating Character

```
import java.util.HashMap;

public class Main {
    static int firstNonRepeating(String s) {
        HashMap<Character, Integer> map = new HashMap<>();

        for (char ch : s.toCharArray()) {
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        for (int i = 0; i < s.length(); i++) {
            if (map.get(s.charAt(i)) == 1) {
                return i;
            }
        }

        return -1;
    }

    public static void main(String[] args) {
        String s = "swiss";

        System.out.println(firstNonRepeating(s));
    }
}
```

12. Valid Anagram

```
import java.util.HashMap;

public class Main {
    static boolean isAnagram(String a, String b) {
        if (a.length() != b.length()) {
            return false;
        }

        HashMap<Character, Integer> map = new HashMap<>();

        for (char ch : a.toCharArray()) {
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }

        for (char ch : b.toCharArray()) {
            if (!map.containsKey(ch)) {
                return false;
            }

            map.put(ch, map.get(ch) - 1);

            if (map.get(ch) < 0) {
                return false;
            }
        }

        return true;
    }

    public static void main(String[] args) {
        String a = "listen";
        String b = "silent";

        System.out.println(isAnagram(a, b));
    }
}
```

13. Longest Substring Without Repeating Characters

```
import java.util.HashSet;

public class Main {
    static int longestSubstring(String s) {
        HashSet<Character> set = new HashSet<>();

        int left = 0;
        int maxLength = 0;

        for (int right = 0; right < s.length(); right++) {
            while (set.contains(s.charAt(right))) {
                set.remove(s.charAt(left));
                left++;
            }

            set.add(s.charAt(right));

            maxLength = Math.max(
                maxLength,
                right - left + 1
            );
        }

        return maxLength;
    }

    public static void main(String[] args) {
        String s = "abcabcbb";

        System.out.println(longestSubstring(s));
    }
}
```

14. Group Anagrams

```
import java.util.*;

public class Main {
    static List<List<String>> groupAnagrams(String[] words) {
        HashMap<String, List<String>> map = new HashMap<>();

        for (String word : words) {
            char[] chars = word.toCharArray();
            Arrays.sort(chars);

            String key = new String(chars);

            if (!map.containsKey(key)) {
                map.put(key, new ArrayList<>());
            }

            map.get(key).add(word);
        }

        return new ArrayList<>(map.values());
    }

    public static void main(String[] args) {
        String[] words = {
            "eat", "tea", "tan", "ate", "nat", "bat"
        };

        System.out.println(groupAnagrams(words));
    }
}
```

15. String Rotation Check

```
public class Main {
    static boolean isRotation(String a, String b) {
        if (a.length() != b.length()) {
            return false;
        }

        String combined = a + a;

        return combined.contains(b);
    }

    public static void main(String[] args) {
        String a = "abcd";
        String b = "cdab";

        System.out.println(isRotation(a, b));
    }
}
```

16. Roman to Integer

```
import java.util.HashMap;

public class Main {
    static int romanToInteger(String s) {
        HashMap<Character, Integer> map = new HashMap<>();

        map.put('I', 1);
        map.put('V', 5);
        map.put('X', 10);
        map.put('L', 50);
        map.put('C', 100);
        map.put('D', 500);
        map.put('M', 1000);

        int result = 0;

        for (int i = 0; i < s.length(); i++) {
            int current = map.get(s.charAt(i));

            if (i + 1 < s.length()
                && current < map.get(s.charAt(i + 1))) {
                result -= current;
            } else {
                result += current;
            }
        }

        return result;
    }

    public static void main(String[] args) {
        String s = "MCMIV";

        System.out.println(romanToInteger(s));
    }
}
```

17. Longest Common Prefix

```
public class Main {
    static String longestCommonPrefix(String[] words) {
        if (words.length == 0) {
            return "";
        }

        String prefix = words[0];

        for (int i = 1; i < words.length; i++) {
            while (!words[i].startsWith(prefix)) {
                prefix = prefix.substring(0, prefix.length() - 1);

                if (prefix.length() == 0) {
                    return "";
                }
            }
        }

        return prefix;
    }

    public static void main(String[] args) {
        String[] words = {
            "flower", "flow", "flight"
        };

        System.out.println(longestCommonPrefix(words));
    }
}
```

18. Remove Adjacent Duplicates

```
import java.util.Stack;

public class Main {
    static String removeDuplicates(String s) {
        Stack<Character> stack = new Stack<>();

        for (char ch : s.toCharArray()) {
            if (!stack.isEmpty() && stack.peek() == ch) {
                stack.pop();
            } else {
                stack.push(ch);
            }
        }

        StringBuilder result = new StringBuilder();

        for (char ch : stack) {
            result.append(ch);
        }

        return result.toString();
    }

    public static void main(String[] args) {
        String s = "abbaca";

        System.out.println(removeDuplicates(s));
    }
}
```

19. Reverse Words in a Sentence

```
public class Main {
    static String reverseWords(String s) {
        String[] words = s.trim().split("\\s+");

        StringBuilder result = new StringBuilder();

        for (int i = words.length - 1; i >= 0; i--) {
            result.append(words[i]);

            if (i != 0) {
                result.append(" ");
            }
        }

        return result.toString();
    }

    public static void main(String[] args) {
        String s = "the sky is blue";

        System.out.println(reverseWords(s));
    }
}
```

20. Count and Say Sequence

```
public class Main {
    static String countAndSay(int n) {
        String result = "1";

        for (int i = 2; i <= n; i++) {
            StringBuilder next = new StringBuilder();

            int count = 1;

            for (int j = 1; j < result.length(); j++) {
                if (result.charAt(j) == result.charAt(j - 1)) {
                    count++;
                } else {
                    next.append(count);
                    next.append(result.charAt(j - 1));
                    count = 1;
                }
            }

            next.append(count);
            next.append(result.charAt(result.length() - 1));

            result = next.toString();
        }

        return result;
    }

    public static void main(String[] args) {
        int n = 5;

        System.out.println(countAndSay(n));
    }
}
```